

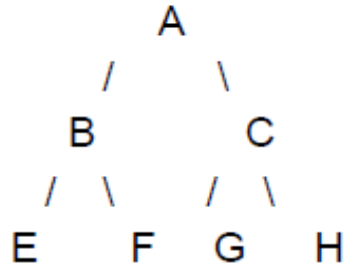
# Thread synchronization 2



Which of the following statements is NOT correct about DFS and BFS?

- ☐ Both DFS and BFS produce the same output.
- ☐ Both DFS and BFS provide a sequence of nodes which will be visited in a graph.
- ☐ DFS is more memory efficient than BFS.
- ☐ They both have high run complexity.

In the graph below:



Which of the followings is a possible visiting list of Depth-First Search (DFS) algorithm?

**Select one alternative:**

- ☐ ABCEFGH
- ☐ ACBEFGH
- ☐ ABCHGFE
- ☐ ABEFCGH

2. Assume there are two problems, each of which can be solved by using a parallel program. Problem 1 has the run complexity of  $\mathcal{O}(n \log n)$  while problem 2 has run complexity of  $\mathcal{O}(n^2)$ . When the data sizes for both problems are the same and are sufficiently large so that solving them by a parallel program is beneficial, which of the following statements is correct (suppose the developer has reasonable skill) ?

**Select one alternative:**

- ☐ Run complexity and data size of each problem do not affect the speedups Problem 1 and Problem 2.
- ☐ The parallel programs that solve problems 1 and 2 have the same speedup.
- ☐ The parallel program that solves problem 2 has a higher speedup than the parallel program that solves problem 1.
- ☐ The parallel program that solves problem 1 has a higher speedup than the parallel program that solves problem 2.

# Conditional Variables

## Conditional Variable

- Create a **critical section**
- A thread enters the critical section under a given **condition**

## Conditional Variable

- Conditional variable has to be used with **Lock** to create a **critical section** and ensure **mutual exclusion**
- A **conditional variable** is needed

## Conditional Variable

- Conditional variable has to be used with **Lock** to create a **critical section** and ensure **mutual exclusion**
- A **conditional variable** is needed



# Implementation

## ➤ Lock to and condition

6 usages

```
private final ReentrantLock lock;
```

5 usages

```
private final Condition condition;
```

7 usages

## Implementation

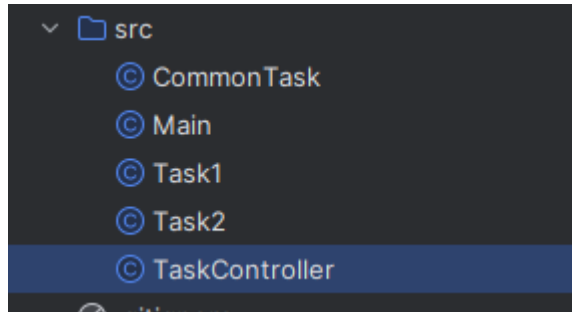
### ➤ Lock to and condition

```
6 usages  
private final ReentrantLock lock;  
5 usages  
private final Condition condition;  
7 usages
```

### ➤ Conditional variable

```
private int turn;
```

# Provided code



```
public void Task1() {
    lock.lock();
    try {
        while (turn != 1)
            condition.await();
        System.out.println("The value of conditional variable is " + turn);
        System.out.println("Thread 1 " + ": Running the task");
        CommonTask.Task();
        System.out.println("Thread 1 " + ": Finishing the execution");
        turn = 2;
        condition.signalAll();
    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        lock.unlock();
    }
}

1 usage
public void Task2() {
    lock.lock();
    try {
        while (turn != 2)
            condition.await();
        System.out.println("The value of conditional variable is " + turn);
        System.out.println("Thread 2: " + ": Running the task");
        CommonTask.Task();
        System.out.println("Thread 2: " + ": Finishing the execution");
        turn = 1;
        condition.signalAll();
    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        lock.unlock();
    }
}
```

```
public TaskController() {
    lock = new ReentrantLock();
    condition = lock.newCondition();
    turn = 1;
}

public void Task1() {
    lock.lock();

    try {

        while (turn != 1)
            condition.await();

        System.out.println("Task1 " + ": Running the task");
        CommonTask.Task();
        System.out.println("Task1 " + ": Finishing the execution\n");

        turn = 2;
        condition.signalAll();

    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        lock.unlock();
    }
}
```

**lock() and unlock() have to be used**

```
public TaskController() {  
    lock = new ReentrantLock();  
    condition = lock.newCondition();  
    turn = 1;  
}  
  
public void Task1() {  
    lock.lock();  
  
    try {  
        while (turn != 1)  
            condition.await();  
  
        System.out.println("Task1 " + ": Running the task");  
        CommonTask.Task();  
        System.out.println("Task1 " + ": Finishing the execution\n");  
  
        turn = 2;  
        condition.signalAll();  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    } finally {  
        lock.unlock();  
    }  
}
```

Conditional variable

```
public TaskController() {
    lock = new ReentrantLock();
    condition = lock.newCondition();
    turn = 1;
}

public void Task1() {
    lock.lock();

    try {

        while (turn != 1)
            condition.await();

        System.out.println("Task1 " + ": Running the task");
        CommonTask.Task();
        System.out.println("Task1 " + ": Finishing the execution\n");

        turn = 2;
        condition.signalAll();

    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        lock.unlock();
    }
}
```

**A thread enters the critical section  
with a given condition**

# Synchronization

- Execution order
  - Task 1
  - Task 2
- User a variable **turn** to synchronize
  - If **turn** = 1, execute Task 1
  - If **turn** = 2, execute Task 2

# Task 1

```
public TaskController() {  
    lock = new ReentrantLock();  
    condition = lock.newCondition();  
    turn = 1;  
}
```

Set turn equal 1 to run Task 1 first

```
public void Task1() {  
    lock.lock();  
  
    try {  
  
        while (turn != 1)  
            condition.await();  
  
        System.out.println("Task1 " + ": Running the task");  
        CommonTask.Task();  
        System.out.println("Task1 " + ": Finishing the execution\n");  
  
        turn = 2;  
        condition.signalAll();  
  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    } finally {  
        lock.unlock();  
    }  
}
```



# Task 1

```
public TaskController() {  
    lock = new ReentrantLock();  
    condition = lock.newCondition();  
    turn = 1;  
}
```

Set turn equal 1 to run Task 1 first

```
public void Task1() {  
    lock.lock();  
  
    try {  
  
        while (turn != 1)  
            condition.await();  
  
        System.out.println("Task1 " + ": Running the task");  
        CommonTask.Task();  
        System.out.println("Task1 " + ": Finishing the execution\n");  
  
        turn = 2;  
        condition.signalAll();  
  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    } finally {  
        lock.unlock();  
    }  
}
```

If turn not equal 1, Task 1 waits.

# Task 1

```
public TaskController() {  
    lock = new ReentrantLock();  
    condition = lock.newCondition();  
    turn = 1;  
}
```

Set turn equal 1 to run Task 1 first

```
public void Task1() {  
    lock.lock();
```

```
    try {
```

```
        while (turn != 1)
```

```
            condition.await();
```

If turn not equal 1, Task 1 waits.

```
        System.out.println("Task1 " + ": Running the task");  
        CommonTask.Task();  
        System.out.println("Task1 " + ": Finishing the execution\n");
```

Critical section

```
        turn = 2;
```

```
        condition.signalAll();
```

```
    } catch (InterruptedException e) {  
        e.printStackTrace();
```

```
    } finally {
```

```
        lock.unlock();
```

```
    }
```

```
}
```

# Task 1

```
public TaskController() {  
    lock = new ReentrantLock();  
    condition = lock.newCondition();  
    turn = 1;  
}
```

Set **turn** equal 1 to run Task 1 first

```
public void Task1() {  
    lock.lock();
```

```
    try {
```

```
        while (turn != 1)  
            condition.await();
```

If **turn** not equal 1, Task 1 waits.

```
        System.out.println("Task1 " + ": Running the task");  
        CommonTask.Task();  
        System.out.println("Task1 " + ": Finishing the execution\n");
```

Critical section

```
        turn = 2;  
        condition.signalAll();
```

Set **turn** equal 2, so that Task 2 can have chance to be executed.

```
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    } finally {  
        lock.unlock();  
    }  
}
```

# Task 1

```
public TaskController() {  
    lock = new ReentrantLock();  
    condition = lock.newCondition();  
    turn = 1;  
}
```

Set **turn** equal 1 to run Task 1 first

```
public void Task1() {  
    lock.lock();
```

```
    try {
```

```
        while (turn != 1)  
            condition.await();
```

If **turn** not equal 1, Task 1 waits.

```
        System.out.println("Task1 " + ": Running the task");  
        CommonTask.Task();  
        System.out.println("Task1 " + ": Finishing the execution\n");
```

Critical section

```
        turn = 2;  
        condition.signalAll();
```

Set **turn** equal 2, so that Task 2 can have chance to be executed.

```
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    } finally {  
        lock.unlock();  
    }  
}
```

Signal Task 2 which is waiting (or any other waiting tasks/threads)

# Task 2

```
public void Task2() {
```

```
    lock.lock();
```

```
    try {
```

```
        while (turn != 2)
            condition.await();
```

← If **turn** not equal 2, Task 2 will not be executed

```
        System.out.println("Task2: " + " : Running the task");
        CommonTask.Task();
        System.out.println("Task2: " + " : Finishing the execution\n");
```

← Critical section

```
        turn = 1;
        condition.signalAll();
```

← Set **turn** equal 1, so that Task 1 can have chance to be executed.

```
    } catch (InterruptedException e) {
        e.printStackTrace();
```

```
    } finally {
        lock.unlock();
```

```
    }
```

```
}
```

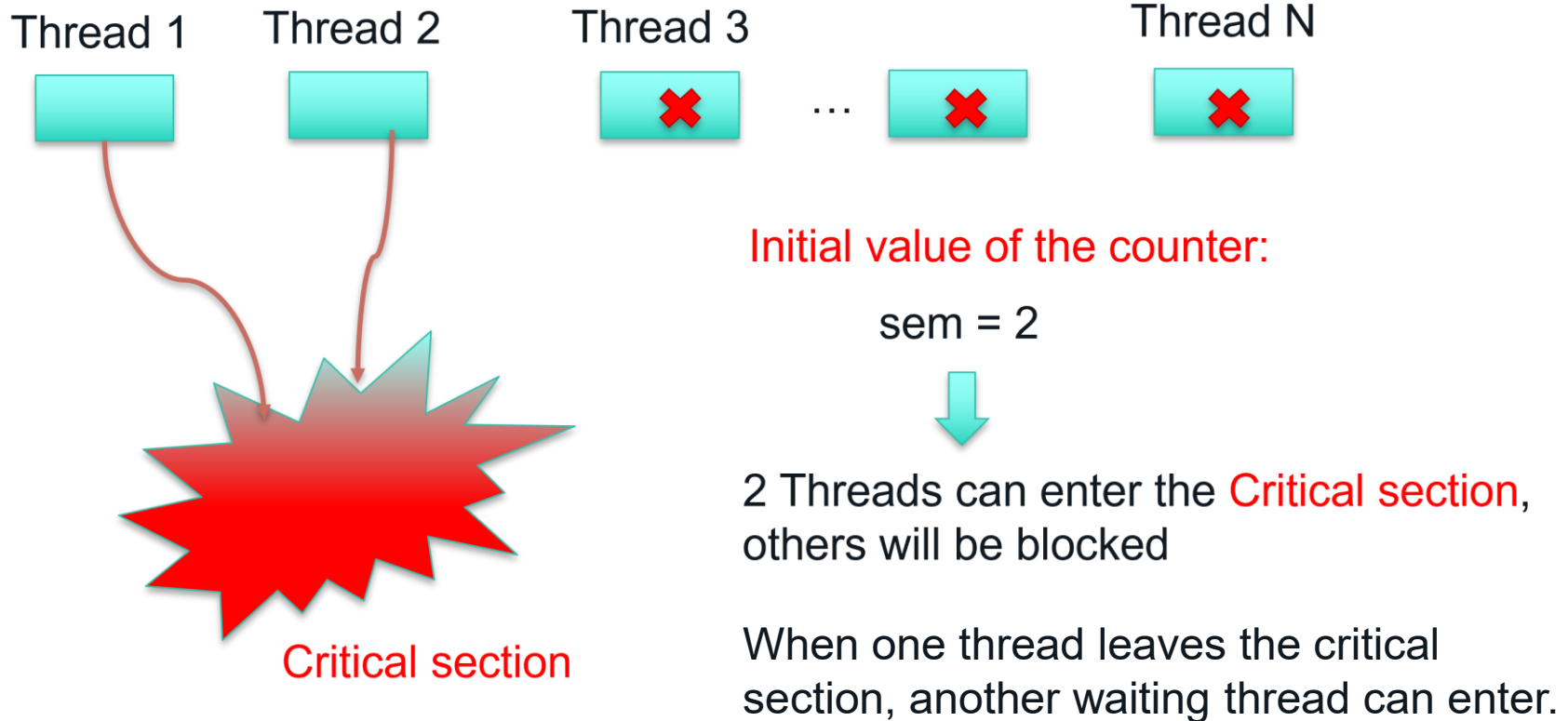
# Semaphores



## Semaphores

- **Multiple** threads can enter a **critical section** simultaneously
- Implementation
  - Create a critical section
  - Control the number of threads which can enter the critical section
    - The number of threads is setup in initial value of **counter**

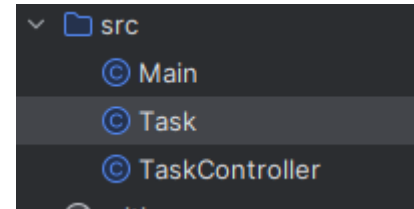
# Semaphores





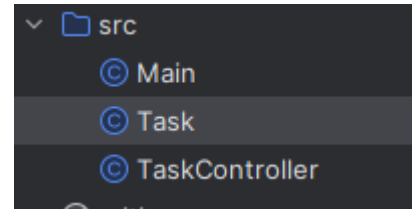
# Semaphores

```
public class Task implements Runnable{  
    private Semaphore semaphore;  
    public Task(Semaphore semaphore) {  
        this.semaphore=semaphore;  
    }  
    public void run() {  
        try {  
            semaphore.acquire();  
            TaskController.Task();  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        } finally {  
            semaphore.release();  
        }  
    }  
}
```



# Semaphores

```
public class Task implements Runnable{  
    private Semaphore semaphore;  
    public Task(Semaphore semaphore) {  
        this.semaphore=semaphore;  
    }  
    public void run() {  
        try {  
            semaphore.acquire();  
            TaskController.Task();  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        } finally {  
            semaphore.release();  
        }  
    }  
}
```



Critical section

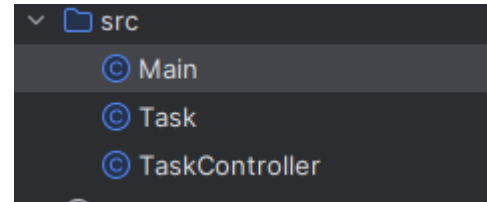
```
public static void Task() {  
    long duration = ThreadLocalRandom.current().nextLong( bound: 10);  
    System.out.printf("Date: "+ new Date() + " %s: Working %d seconds\n",Thread.currentThread().getName(),duration);  
    try {  
        TimeUnit.SECONDS.sleep(duration);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
}
```

  Critical section

# Semaphores

```
public static void main(String[] args) {  
    Semaphore semaphore=new Semaphore(3);  
    ThreadPoolExecutor executor=(ThreadPoolExecutor)Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors());  
  
    for (int i=0; i<10; i++) {  
        executor.execute(new Task(semaphore));  
    }  
  
    executor.shutdown();  
    try {  
        executor.awaitTermination(1, TimeUnit.DAYS);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
}
```

← counter



# Semaphores

```
Date: Wed Feb 25 13:02:14 CET 2026 pool-1-thread-1: Working 4 seconds
Date: Wed Feb 25 13:02:18 CET 2026 pool-1-thread-3: Working 5 seconds
Date: Wed Feb 25 13:02:23 CET 2026 pool-1-thread-3: Working 2 seconds
Date: Wed Feb 25 13:02:25 CET 2026 pool-1-thread-4: Working 7 seconds
Date: Wed Feb 25 13:02:32 CET 2026 pool-1-thread-2: Working 6 seconds
Date: Wed Feb 25 13:02:38 CET 2026 pool-1-thread-5: Working 1 seconds
Date: Wed Feb 25 13:02:39 CET 2026 pool-1-thread-6: Working 3 seconds
Date: Wed Feb 25 13:02:42 CET 2026 pool-1-thread-7: Working 2 seconds
Date: Wed Feb 25 13:02:44 CET 2026 pool-1-thread-8: Working 4 seconds
Date: Wed Feb 25 13:02:48 CET 2026 pool-1-thread-1: Working 9 seconds
```

Output when the initial value of counter is 1. Semaphore is the same as ReentrantLock.

# Semaphores

```
Date: Wed Feb 25 13:03:54 CET 2026 pool-1-thread-2: Working 8 seconds
Date: Wed Feb 25 13:03:54 CET 2026 pool-1-thread-1: Working 6 seconds
Date: Wed Feb 25 13:04:00 CET 2026 pool-1-thread-3: Working 4 seconds
Date: Wed Feb 25 13:04:02 CET 2026 pool-1-thread-2: Working 3 seconds
Date: Wed Feb 25 13:04:04 CET 2026 pool-1-thread-4: Working 2 seconds
Date: Wed Feb 25 13:04:05 CET 2026 pool-1-thread-5: Working 6 seconds
Date: Wed Feb 25 13:04:06 CET 2026 pool-1-thread-6: Working 2 seconds
Date: Wed Feb 25 13:04:08 CET 2026 pool-1-thread-7: Working 3 seconds
Date: Wed Feb 25 13:04:11 CET 2026 pool-1-thread-8: Working 5 seconds
Date: Wed Feb 25 13:04:11 CET 2026 pool-1-thread-1: Working 1 seconds
```

Output when the initial value of counter is 2. **2 Threads** are executed at the same time.

# Semaphores

```
Date: Wed Feb 25 13:09:33 CET 2026 pool-1-thread-3: Working 5 seconds
Date: Wed Feb 25 13:09:33 CET 2026 pool-1-thread-2: Working 3 seconds
Date: Wed Feb 25 13:09:33 CET 2026 pool-1-thread-1: Working 7 seconds
Date: Wed Feb 25 13:09:37 CET 2026 pool-1-thread-4: Working 6 seconds
Date: Wed Feb 25 13:09:39 CET 2026 pool-1-thread-3: Working 0 seconds
Date: Wed Feb 25 13:09:39 CET 2026 pool-1-thread-5: Working 5 seconds
Date: Wed Feb 25 13:09:41 CET 2026 pool-1-thread-6: Working 8 seconds
Date: Wed Feb 25 13:09:43 CET 2026 pool-1-thread-7: Working 7 seconds
Date: Wed Feb 25 13:09:44 CET 2026 pool-1-thread-8: Working 5 seconds
Date: Wed Feb 25 13:09:49 CET 2026 pool-1-thread-2: Working 9 seconds
```

Output when the initial value of counter is 3. **3 Threads** are executed at the same time.

# Semaphores

```
Date: Wed Feb 25 13:11:39 CET 2026 pool-1-thread-4: Working 7 seconds
Date: Wed Feb 25 13:11:39 CET 2026 pool-1-thread-1: Working 3 seconds
Date: Wed Feb 25 13:11:39 CET 2026 pool-1-thread-3: Working 1 seconds
Date: Wed Feb 25 13:11:39 CET 2026 pool-1-thread-2: Working 9 seconds
Date: Wed Feb 25 13:11:40 CET 2026 pool-1-thread-5: Working 4 seconds
Date: Wed Feb 25 13:11:42 CET 2026 pool-1-thread-1: Working 0 seconds
Date: Wed Feb 25 13:11:42 CET 2026 pool-1-thread-7: Working 0 seconds
Date: Wed Feb 25 13:11:42 CET 2026 pool-1-thread-6: Working 4 seconds
Date: Wed Feb 25 13:11:44 CET 2026 pool-1-thread-8: Working 4 seconds
Date: Wed Feb 25 13:11:46 CET 2026 pool-1-thread-3: Working 7 seconds
```

Output when the initial value of counter is 4. 4 Threads are executed at the same time.



# Semaphores

```
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-5: Working 3 seconds
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-3: Working 8 seconds
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-1: Working 6 seconds
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-4: Working 8 seconds
Date: Wed Feb 25 13:13:44 CET 2026 pool-1-thread-2: Working 7 seconds
Date: Wed Feb 25 13:13:47 CET 2026 pool-1-thread-5: Working 9 seconds
Date: Wed Feb 25 13:13:50 CET 2026 pool-1-thread-6: Working 1 seconds
Date: Wed Feb 25 13:13:51 CET 2026 pool-1-thread-7: Working 8 seconds
Date: Wed Feb 25 13:13:51 CET 2026 pool-1-thread-8: Working 5 seconds
Date: Wed Feb 25 13:13:52 CET 2026 pool-1-thread-1: Working 1 seconds
```

Output when the initial value of counter is 5. **5 Threads** are executed at the same time.

# Semaphores

```
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-1: Working 2 seconds  
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-3: Working 9 seconds  
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-2: Working 1 seconds
```

3 threads execute at  
the same time

```
Date: Wed Feb 25 15:06:29 CET 2026 pool-1-thread-2: Finished  
Date: Wed Feb 25 15:06:29 CET 2026 pool-1-thread-7: Working 9 seconds  
Date: Wed Feb 25 15:06:30 CET 2026 pool-1-thread-1: Finished  
Date: Wed Feb 25 15:06:30 CET 2026 pool-1-thread-1: Working 2 seconds  
Date: Wed Feb 25 15:06:32 CET 2026 pool-1-thread-1: Finished  
Date: Wed Feb 25 15:06:32 CET 2026 pool-1-thread-8: Working 4 seconds  
Date: Wed Feb 25 15:06:36 CET 2026 pool-1-thread-8: Finished  
Date: Wed Feb 25 15:06:36 CET 2026 pool-1-thread-6: Working 8 seconds  
Date: Wed Feb 25 15:06:37 CET 2026 pool-1-thread-3: Finished  
Date: Wed Feb 25 15:06:37 CET 2026 pool-1-thread-4: Working 5 seconds  
Date: Wed Feb 25 15:06:38 CET 2026 pool-1-thread-7: Finished  
Date: Wed Feb 25 15:06:38 CET 2026 pool-1-thread-5: Working 2 seconds  
Date: Wed Feb 25 15:06:40 CET 2026 pool-1-thread-5: Finished  
Date: Wed Feb 25 15:06:40 CET 2026 pool-1-thread-2: Working 6 seconds  
Date: Wed Feb 25 15:06:42 CET 2026 pool-1-thread-4: Finished  
Date: Wed Feb 25 15:06:44 CET 2026 pool-1-thread-6: Finished  
Date: Wed Feb 25 15:06:46 CET 2026 pool-1-thread-2: Finished
```

Another output when the counter is 3.

# Semaphores

```
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-1: Working 2 seconds
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-3: Working 9 seconds
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-2: Working 1 seconds
```

3 threads execute at  
the same time

One thread finishes  
in critical section

```
Date: Wed Feb 25 15:06:29 CET 2026 pool-1-thread-2: Finished
Date: Wed Feb 25 15:06:29 CET 2026 pool-1-thread-7: Working 9 seconds
Date: Wed Feb 25 15:06:30 CET 2026 pool-1-thread-1: Finished
Date: Wed Feb 25 15:06:30 CET 2026 pool-1-thread-1: Working 2 seconds
Date: Wed Feb 25 15:06:32 CET 2026 pool-1-thread-1: Finished
Date: Wed Feb 25 15:06:32 CET 2026 pool-1-thread-8: Working 4 seconds
Date: Wed Feb 25 15:06:36 CET 2026 pool-1-thread-8: Finished
Date: Wed Feb 25 15:06:36 CET 2026 pool-1-thread-6: Working 8 seconds
Date: Wed Feb 25 15:06:37 CET 2026 pool-1-thread-3: Finished
Date: Wed Feb 25 15:06:37 CET 2026 pool-1-thread-4: Working 5 seconds
Date: Wed Feb 25 15:06:38 CET 2026 pool-1-thread-7: Finished
Date: Wed Feb 25 15:06:38 CET 2026 pool-1-thread-5: Working 2 seconds
Date: Wed Feb 25 15:06:40 CET 2026 pool-1-thread-5: Finished
Date: Wed Feb 25 15:06:40 CET 2026 pool-1-thread-2: Working 6 seconds
Date: Wed Feb 25 15:06:42 CET 2026 pool-1-thread-4: Finished
Date: Wed Feb 25 15:06:44 CET 2026 pool-1-thread-6: Finished
Date: Wed Feb 25 15:06:46 CET 2026 pool-1-thread-2: Finished
```

Another output when the counter is 3.

# Semaphores

```
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-1: Working 2 seconds
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-3: Working 9 seconds
Date: Wed Feb 25 15:06:28 CET 2026 pool-1-thread-2: Working 1 seconds
```

3 threads execute at the same time

One thread finishes in critical section

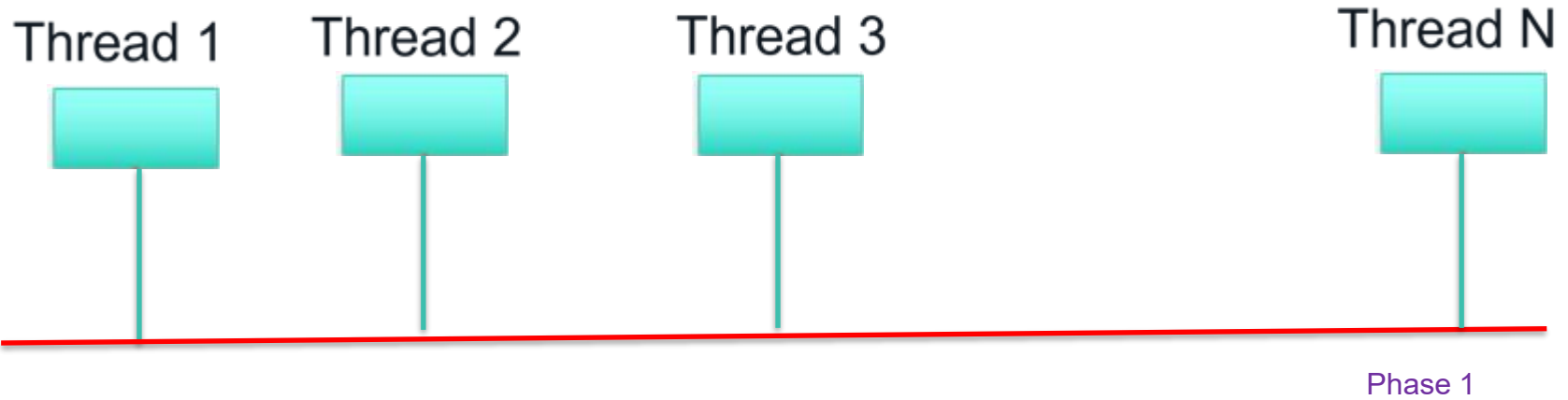
```
Date: Wed Feb 25 15:06:29 CET 2026 pool-1-thread-2: Finished
Date: Wed Feb 25 15:06:29 CET 2026 pool-1-thread-7: Working 9 seconds
Date: Wed Feb 25 15:06:30 CET 2026 pool-1-thread-1: Finished
Date: Wed Feb 25 15:06:30 CET 2026 pool-1-thread-1: Working 2 seconds
Date: Wed Feb 25 15:06:32 CET 2026 pool-1-thread-1: Finished
Date: Wed Feb 25 15:06:32 CET 2026 pool-1-thread-8: Working 4 seconds
Date: Wed Feb 25 15:06:36 CET 2026 pool-1-thread-8: Finished
Date: Wed Feb 25 15:06:36 CET 2026 pool-1-thread-6: Working 8 seconds
Date: Wed Feb 25 15:06:37 CET 2026 pool-1-thread-3: Finished
Date: Wed Feb 25 15:06:37 CET 2026 pool-1-thread-4: Working 5 seconds
Date: Wed Feb 25 15:06:38 CET 2026 pool-1-thread-7: Finished
Date: Wed Feb 25 15:06:38 CET 2026 pool-1-thread-5: Working 2 seconds
Date: Wed Feb 25 15:06:40 CET 2026 pool-1-thread-5: Finished
Date: Wed Feb 25 15:06:40 CET 2026 pool-1-thread-2: Working 6 seconds
Date: Wed Feb 25 15:06:42 CET 2026 pool-1-thread-4: Finished
Date: Wed Feb 25 15:06:44 CET 2026 pool-1-thread-6: Finished
Date: Wed Feb 25 15:06:46 CET 2026 pool-1-thread-2: Finished
```

Another thread can enter the critical section

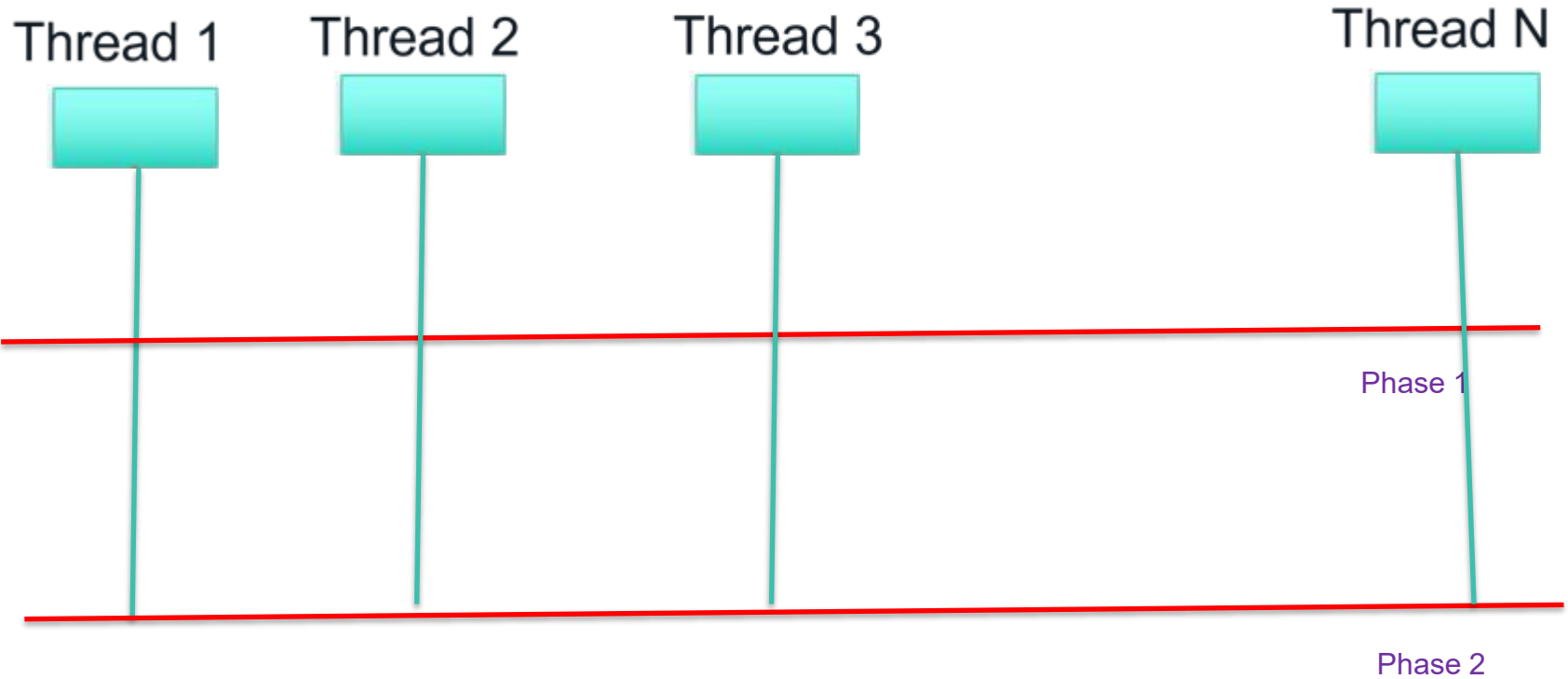
Another output when the counter is 3.

Barrier





All threads meet Phase 1 before any one can move further



# Barrier Task

```
public void run() {  
    System.out.println(Thread.currentThread().getName()+": Phase 1");  
    CommonTask.doTask();  
    try {  
        barrier.await();  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    } catch (BrokenBarrierException e) {  
        e.printStackTrace();  
    }  
    System.out.println(Thread.currentThread().getName()+": Phase 2");  
}
```



# Main

```
public class Main {  
    public static void main(String[] args) {  
        CyclicBarrier barrier = new CyclicBarrier(10, new FinishBarrierTask());  
  
        ThreadPoolExecutor executor=(ThreadPoolExecutor)Executors.newCachedThreadPool();  
  
        for (int i=0; i<10; i++) {  
            executor.execute(new Task(barrier));  
        }  
  
        executor.shutdown();  
  
        try {  
            executor.awaitTermination(1, TimeUnit.DAYS);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

## References

- Maurice Herlihy & Nir Shavit, “The Art of Multiprocessor Programming”, Elsevier, 2008.
- Gonzalez, “Javier Fernandez. Java 9 Concurrency Cookbook”, Packt Publishing Ltd, 2017.